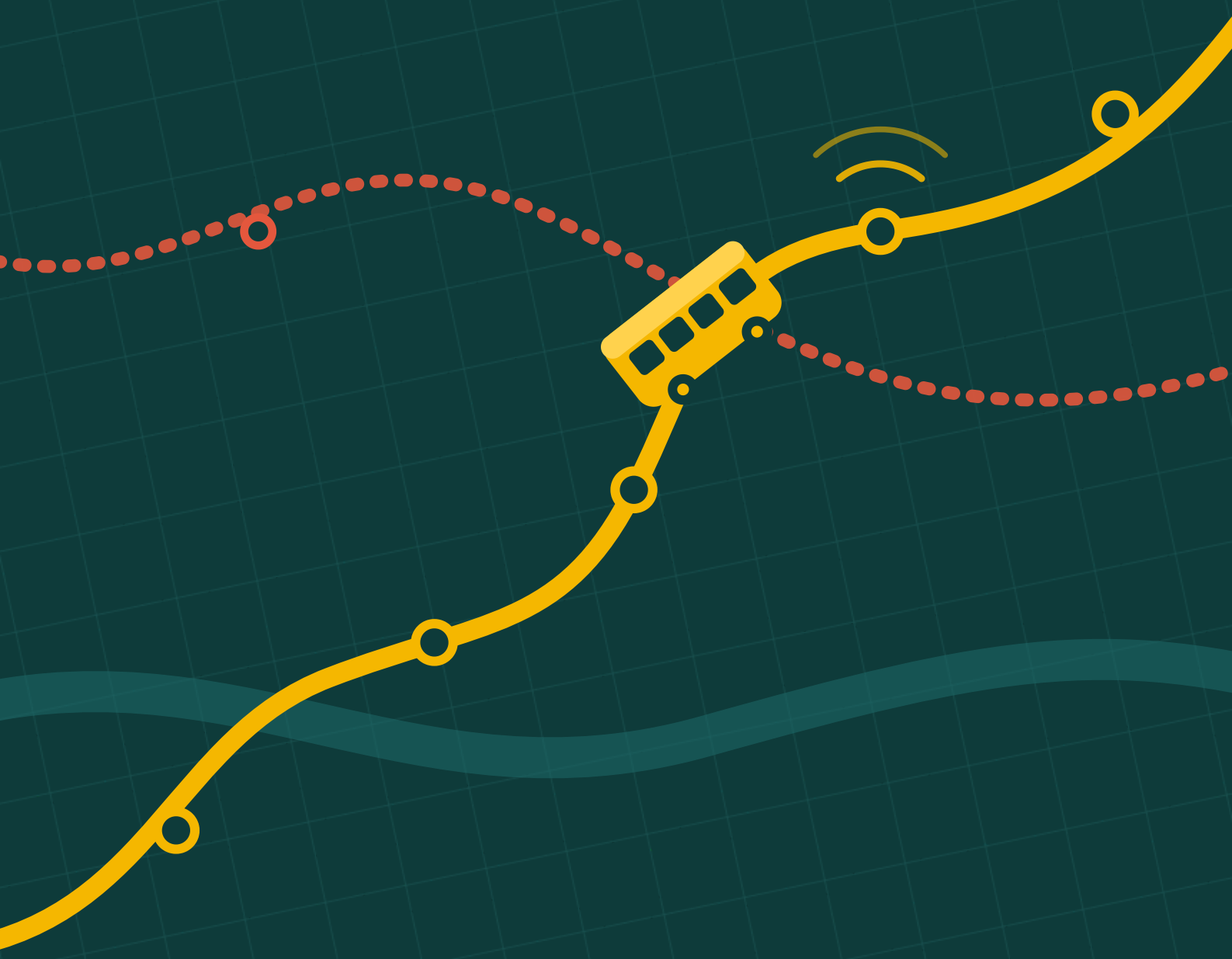
ROADMAP DEL PROYECTO

Bus Portoviejo

Del prototipo a una app para la ciudad: saber dónde viene tu bus y si te da tiempo de llegar a la parada.



Manabí, Ecuador
Noviembre de 2026 · Documento para el equipo

App Android
React Native · Django · MapLibre

LA IDEA EN UNA PÁGINA

Que esperar el bus deje de ser adivinar.

Hoy en Portoviejo nadie sabe cuánto falta para que pase el bus: se sale de casa a ciegas y se espera en la parada sin saber si ya pasó. **Bus Portoviejo** muestra los buses urbanos en el mapa en tiempo real y te dice si te da tiempo o si tienes que correr.

Qué hará la app

- **Buses en vivo:** ver en el mapa dónde va el bus de tu línea.
- **Tiempo de llegada:** cuánto falta y un aviso *Tranquilo / Sal ya / No llegas*.
- **Viaje A → B:** qué línea tomar, incluso con un transbordo.
- **Avisos:** una notificación cuando tu bus está cerca.

Lo que ya existe

- Una demo con el **mapa real de Portoviejo** en vista isométrica, con 7.600 edificios en 3D.
- Una línea de ejemplo con 10 paradas, 3 buses simulados y el cálculo de llegada.
- Una prueba en vivo con un celular haciendo de GPS.

La línea de ejemplo no existe. El siguiente paso es usar líneas reales.

La meta

Una **app Android instalable** con **2 o 3 líneas reales**, buses simulados moviéndose en vivo, tiempo de llegada, viaje A→B y avisos, más un panel web para administrar líneas y paradas. Con eso se va a presentar el proyecto a **las autoridades y las cooperativas**, junto con un plan piloto. Si lo financian, ellos ponen los GPS físicos en los buses.

Con qué se construye

React Native + Expo · Django · Django Channels · WebSockets · MapLibre · Android primero · Servicios gratuitos para la demo · Datos en GTFS

Por qué vale la pena

01

Un problema de todos los días

Cualquiera que tome bus en Portoviejo lo entiende en cinco segundos.

02

Ya hay algo que mostrar

Un mapa que funciona convence más que una idea en una diapositiva.

03

Sin gastar en hardware

Primero se demuestra con simulación. Los GPS los pone quien financie.

EL RECORRIDO COMPLETO

Diez paradas hasta la ciudad

Cada fase es una parada. No hay fechas fijas: se avanza en orden y no se empieza una fase sin terminar la anterior. Solo el trabajo de campo va en paralelo.

0 — Preparar el terreno — PROGRAMACIÓN
Herramientas, repositorio nuevo y tablero de tareas.

1 — Datos reales de 2 o 3 rutas — CAMPO
Marcar las paradas en uMap y convertirlas a GTFS.

2 — Servidor Django — PROGRAMACIÓN
Líneas, paradas, panel de administración y API.

3 — Simulación en tiempo real — PROGRAMACIÓN
Buses ficticios sobre recorridos reales, enviados por WebSocket.

4 — La app con el mapa — PROGRAMACIÓN
Expo + MapLibre: ver los buses moverse en un Android.

El trabajo de campo sigue mientras se programan las fases 2 a 4

5 — Tiempo de llegada — PROGRAMACIÓN
"Llega en 6 min", más Tranquilo / Sal ya / No llegas.

6 — Viaje A → B — PROGRAMACIÓN
Qué línea tomar, directa o con un transbordo.

7 — Avisos — PROGRAMACIÓN
Notificación cuando el bus está cerca de tu parada.

8 — Presentación a las autoridades — PRESENTACIÓN
APK, video, documento y plan piloto con costos.

9 — GPS reales en los buses — TODOS
Solo si hay financiamiento: hardware, seguridad y Play Store.

Si el tiempo aprieta, este es el orden: Buses en vivo → Tiempo de llegada → Viaje A→B → Avisos

Con las dos primeras ya hay una demo que se puede presentar.

PARADAS 0 Y 1

Preparar y salir a la calle

0–1

0 Preparar el terreno

Dejar todo listo para que cualquiera pueda empezar a trabajar.

PROGRAMACIÓN

- [ ] Antes de borrar el prototipo web, marcarlo con una etiqueta (`git tag prototipo-web`). Su lógica de llegadas y sus scripts sirven de referencia.
- [ ] Reorganizar el repositorio en dos carpetas: `movil/` (Expo) y `servidor/` (Django).
- [ ] Instalar Node.js LTS, Python 3, Git y Android Studio (para el emulador y para compilar).
- [ ] Elegir una licencia, por ejemplo MIT, o dejarlo privado si se piensa licenciar al municipio.
- [ ] Crear un tablero en GitHub (*Projects*) y pasar cada tarea a un *Issue*, para que quien se sume tome una tarea sin preguntar.

Listo cuando: Expo y Django arrancan en tu computadora y el repositorio tiene la estructura nueva.

1 Datos reales de 2 o 3 rutas

Es la base de todo: sin rutas reales, la demo no convence.

CAMPO

- [ ] Elegir 2 o 3 rutas que **se crucen** entre sí (para que el viaje A→B tenga transbordos) y que una de ellas pase por la UTM.
- [ ] Quienes ya conocen las rutas marcan las paradas en **uMap**, un mapa colaborativo sobre OpenStreetMap: una capa por sentido (`Ruta 2 - ida`) y un marcador por parada (`03 Parque Central`).
- [ ] Anotar en `rutas.csv` la frecuencia aproximada, el horario de servicio y la cooperativa de cada ruta.
- [ ] El conversor `convertir_a_gtfs.py` junta las paradas compartidas, traza el recorrido por las calles y genera el **GTFS**, el estándar mundial de transporte público.
- [ ] Revisar la vista previa y validar el GTFS con el validador oficial de MobilityData.
- [ ] Más adelante, corregir los tiempos estimados viajando una vez en cada ruta con **GPSLogger**.

Ojo: Google Maps sirve para acordarse de una ruta, pero sus datos no se pueden copiar: sus términos lo prohíben. Los marcadores salen de lo que ustedes saben y vieron.

Guía: el paso a paso para marcar, exportar y convertir está en `datos/GUIA_UMAP.md` .

Listo cuando: hay paradas, recorridos y frecuencias de 2 o 3 rutas, en GTFS dentro de `datos/gtfs/` , sin errores del validador.

PARADAS 2, 3 Y 4

El motor y el mapa

2–4

2 Servidor Django

La base de datos de líneas y paradas, con un panel para editarlas.

PROGRAMACIÓN

- [ ] Modelos: `Linea`, `Parada`, `Trazado`, `ParadaEnLinea` (orden y tiempo por tramo) y `Bus`.
- [ ] Activar el **panel de administración de Django** para crear y editar líneas sin programar. Conviene mostrárselo a las autoridades.
- [ ] Un comando `importar_gtfs` que cargue los datos de la fase 1.
- [ ] Una API con Django REST Framework: `/api/lineas`, sus paradas y su trazado.
- [ ] Publicarlo gratis (Render, Koyeb o PythonAnywhere). En la mayoría el servidor "se duerme": hay que despertarlo antes de cada demo.

Listo cuando: abres la API desde el celular y devuelve las líneas reales.

3 Simulación en tiempo real

Buses ficticios que se mueven como los de verdad.

PROGRAMACIÓN

- [ ] Un simulador que haga salir buses según la frecuencia de cada línea y los mueva por el trazado real con las velocidades medidas.
- [ ] Variaciones realistas: buses más rápidos o más lentos, esperas distintas en las paradas y "tráfico" en hora pico.
- [ ] Un canal WebSocket por línea (`/ws/lineas/<id>/`) que envíe las posiciones cada 2 a 5 segundos.
- [ ] Para la demo basta la capa en memoria de Channels, sin Redis. Redis llega en la fase 9.
- [ ] Usar para el mensaje la misma forma que tendrá el GPS real (`bus, lat, lon, velocidad, rumbo, hora`). Así, cuando haya GPS físicos, la app no se toca.

Listo cuando: un cliente WebSocket de prueba recibe los buses simulados en movimiento.

4 La app con el mapa

El primer momento en que se ve en un celular.

PROGRAMACIÓN

- [ ] Crear la app con Expo y TypeScript, y agregar `@maplibre/maplibre-react-native`.
- [ ] Ese paquete **no corre en Expo Go**: hace falta un *development build* (`npx expo run:android`). Es la primera traba esperable; no es un error tuyo.
- [ ] Mapa base gratuito (por ejemplo, OpenFreeMap, sin clave y con edificios). La vista 3D se recupera con `fill-extrusion`.
- [ ] Pantallas: lista de líneas → mapa de la línea → buses en vivo. Mostrar "Sin conexión" y reconectar solo.

Listo cuando: en un Android real ves los buses simulados moverse sobre las líneas reales.

PARADAS 5, 6 Y 7

Lo que la gente va a usar

5–7

5 Tiempo de llegada

La función estrella: cuánto falta y si te da tiempo.

PROGRAMACIÓN

- [ ] El servidor calcula cuánto le falta a cada bus para llegar a las paradas siguientes: la distancia sobre el trazado dividida por la velocidad del tramo, más las esperas.
- [ ] En la app: tocas una parada y ves "llega en X min", además de **Tranquilo / Sal ya / No llegas** según lo que tardas caminando (con `expo-location`).
- [ ] Guardar paradas favoritas en el celular.

Listo cuando: la llegada mostrada coincide con lo que hace el simulador.

6 Viaje A → B

Decirle a la gente qué bus tomar.

PROGRAMACIÓN

- [ ] El usuario elige origen y destino tocando el mapa o con su ubicación actual.
- [ ] Con 2 o 3 líneas basta un método simple: paradas a menos de unos 500 m a pie del origen y del destino, primero las líneas directas y después las combinaciones con **un transbordo**.
- [ ] Ordenar las opciones por hora de llegada usando los tiempos en vivo.
- [ ] Más adelante, con muchas líneas, pasar a OpenTripPlanner, que lee GTFS directamente.

Listo cuando: de la UTM al centro, la app propone la mejor opción con su hora de llegada.

7 Avisos

Que el celular te avise, sin estar mirando el mapa.

PROGRAMACIÓN

- [ ] Primero, avisos **locales** con `expo-notifications`: "Tu bus está a 3 minutos de la parada".
- [ ] Después, avisos **push** enviados por el servidor aunque la app esté cerrada. Es lo más difícil: requiere Firebase Cloud Messaging y un servidor siempre despierto, algo que los planes gratuitos no garantizan.

Para la demo: alcanza con los avisos locales. Los push quedan para cuando haya presupuesto.

PARADAS 8 Y 9

Presentarlo y hacerlo real

8–9

8 Presentación a las autoridades

Ideal para quien prefiere la parte no técnica.

PRESENTACIÓN

- [ ] Generar el APK con EAS Build (tiene plan gratuito) e instalarlo en 2 o 3 celulares para la reunión.
- [ ] Grabar un video corto de la app funcionando, por si falla el internet en la sala.
- [ ] Documento y diapositivas con el problema (cuánto espera hoy un pasajero), la solución, la demo y un **plan piloto**: una línea, cuántos buses, el costo de cada GPS con su chip de datos y el costo mensual del servidor.
- [ ] Llevar los **datos GTFS** de las líneas levantadas. Son un valor real para el municipio aunque no compren la app.
- [ ] Usar el contacto informal para conseguir la reunión con la dirección de movilidad o la ATM, y averiguar antes si las cooperativas ya tienen rastreadores.

Listo cuando: se hizo la presentación y hay una respuesta: sí, no o "vuelvan con X".

9 GPS reales en los buses

Solo si hay financiamiento o un piloto aprobado.

TODOS

- [ ] Poner **Traccar Server** como puerta de entrada de los GPS físicos: entiende más de 200 protocolos (Teltonika, Concox...). Django lee de Traccar y reemplaza al simulador sin tocar la app.
- [ ] Seguridad: autenticación de los dispositivos, HTTPS y límites de uso.
- [ ] Redis para Channels y un servidor que no se duerma (para entonces ya habría presupuesto).
- [ ] Medir en la calle la diferencia entre la llegada estimada y la real, y ajustar.
- [ ] Publicar en Play Store (pago único de 25 USD) y después en iPhone (Apple Developer, 99 USD al año).
- [ ] Ampliar a todas las líneas de la ciudad.

El mensaje de posición se diseña desde la fase 3 con la misma forma que mandará el GPS real. Por eso, pasar de la simulación a los buses de verdad es **cambiar de dónde vienen los datos**, no rehacer la app.

PARA SUMARSE AL PROYECTO

¿Te subes?

No hace falta saber programar. Hay tres formas de ayudar y cada una tiene un primer paso claro.

Trabajo de campo

Conoces las rutas de bus de Portoviejo.

1. Elige una ruta que conozcas bien.
2. Abre el mapa del proyecto en uMap.
3. Marca sus paradas, ida y vuelta, en orden.
4. Anota cada cuánto pasa y su horario.

FASE 1 · SIN PROGRAMAR

Presentación

Te va bien hablar, escribir y convencer.

1. Averigua quién decide sobre movilidad en Portoviejo.
2. Junta datos del problema: esperas y quejas.
3. Arma el borrador del plan piloto y sus costos.
4. Prepara las diapositivas.

FASE 8 · SIN PROGRAMAR

Programación

Quieres aprender o ya sabes React Native o Django.

1. Pide acceso al repositorio en GitHub.
2. Elige un *Issue* del tablero.
3. Trabaja en una rama propia.
4. Abre un *pull request* a `main`.

FASES 0 A 7

Riesgos que tenemos presentes

Mucho alcance

Cuatro funciones es bastante. Se sigue el orden de prioridad y con dos ya hay demo.

Mucho que aprender

Expo, Django y Channels a la vez. Se avanza fase por fase, sin saltarse ninguna.

Depender de las autoridades

Si dicen que no, los datos y la app siguen sirviendo: se ofrecen a una cooperativa o a los mismos pasajeros.

Servicios gratuitos

Se duermen o cambian sus condiciones. Siempre hay video de respaldo y se sabe volver a desplegar.

Una rama por tarea y un pull request a `main`.

No subir recorridos GPS sin recortar los tramos cerca de casas.

El bus ya está saliendo.

Si te interesa alguna de estas tareas, escríbeme y te paso el acceso al repositorio. Con que alguien tome una sola línea en la fase 1 ya se avanza muchísimo.

